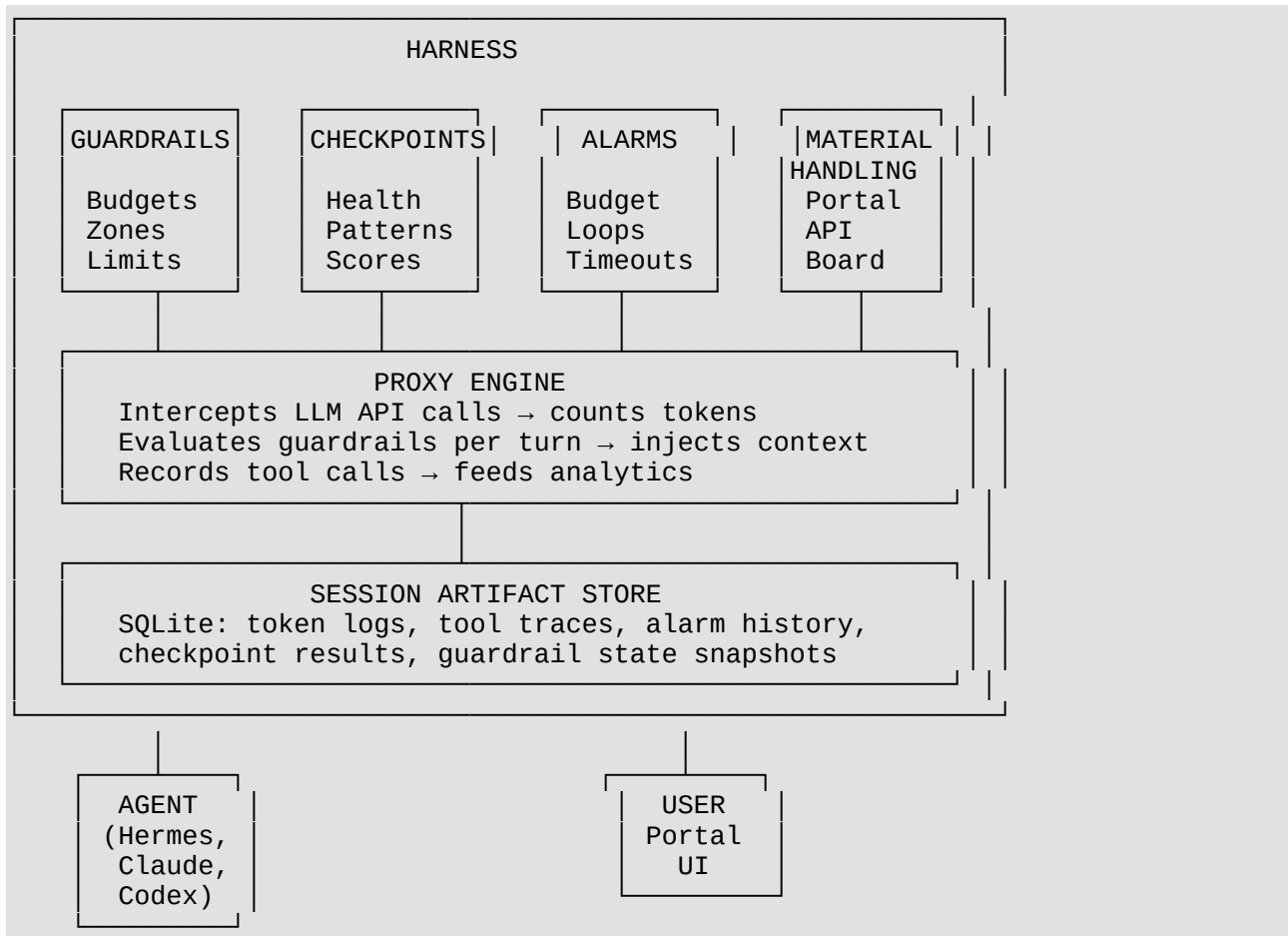


Token-Tracker Harness

A harness that governs AI coding agents through **declared guardrails**, **structured checkpoints**, **clean material handling**, and **named alarms** — all operating in the domain of token-budget governance.

Architecture Overview



Single-process design: The harness runs as one FastAPI application serving three roles:

- LLM Proxy** — The agent points its `OPENAI_BASE_URL` (or equivalent) at the harness. All API calls arrive at the harness, which uses [LiteLLM](#) (`litellm.acompletion()`) to forward to the real provider. LiteLLM normalizes request/response shapes across 100+ providers (OpenAI, Anthropic, Gemini, etc.) — token counts are always `response.usage.prompt_tokens / completion_tokens` regardless of provider. Governance layers (system prompt rewrite, `max_tokens` clamping, tool restriction) are injected into the request before forwarding.
- REST API** — Session management (`/session/start`, `/session/{id}/report`), guardrail CRUD, checkpoint evaluation, alarm history.
- Portal UI** — HTML served via HTMX + Jinja2. Dashboard with token charts, AGILE task board, session history, alarm log.

Deployment: Single Docker container. Runs identically on `localhost` or cloud (Fly.io, Railway). SQLite backs the artifact store — no external database dependency.

Multi-provider support: Because LiteLLM normalizes all providers into a common interface, the harness supports any agent that speaks OpenAI-compatible or Anthropic-native APIs. The proxy endpoint `/v1/chat/completions` forwards via `litellm.acompletion()`, which auto-detects the target provider from the `model` field. Adding a new provider is a configuration change, not a code change.

LiteLLM Integration

LiteLLM is the transport layer — it handles provider abstraction, token counting, cost calculation, and streaming. The harness owns all governance logic and runs it *before* calling LiteLLM.

What LiteLLM provides (free):

Capability	LiteLLM API	Notes
Provider abstraction	<code>litellm.acompletion(model="...", messages=..., ...)</code>	One call for 100+ providers; auto-detects from model name
Token counting	<code>response.usage.prompt_tokens</code> , <code>response.usage.completion_tokens</code>	Normalized across all providers
Cost calculation	<code>litellm.cost_per_token(model=..., prompt_tokens=..., completion_tokens=...)</code>	Returns USD cost — no pricing tables to maintain
Streaming	<code>stream=True</code> , <code>stream_options={"include_usage": True}</code>	Final usage chunk provides accurate total token counts
Response normalization	<code>response.choices[0].message.content</code>	Always OpenAI-shaped, even for Anthropic-native models

What the harness owns (our code):

All governance happens before the LiteLLM call. On every turn:

```
# 1. Check zone state (green/yellow/red based on live daily budget)
zone = session.get_zone()

# 2. Rewrite system prompt (Layer 1)
messages = inject_zone_prompt(original_messages, zone)

# 3. Clamp max_tokens (Layer 2)
max_tokens = ZONE_MAX_TOKENS[zone]

# 4. Strip blocked tools (Layer 3)
tools = filter_blocked_tools(original_tools, zone)

# 5. Forward via LiteLLM
```

```

response = await litellm.acompletion(
    model=session.model,
    messages=messages,
    max_tokens=max_tokens,
    tools=tools,
    stream=True,
    stream_options={"include_usage": True},
)

# 6. Extract token counts, store in session artifact
session.record_turn(response.usage.prompt_tokens,
response.usage.completion_tokens)

# 7. Fire alarms if thresholds crossed
session.check_alarms()

```

Streaming token counting caveat: LiteLLM has a [known issue](#) with inflated `prompt_tokens` when accumulating across streaming chunks. Workaround: use `stream_options={"include_usage": True}` and read token counts exclusively from the final usage chunk, which is accurate. Do not sum across intermediate chunks.

Cost tracking: LiteLLM's built-in cost calculator (`litellm.cost_per_token()`) uses the model pricing table maintained in the `model_prices_and_context_window` library (auto-updated). The harness calls this after every turn to compute the USD cost of that turn, which feeds the portal dashboard and budget tracking.

Why not LiteLLM's built-in guardrails? LiteLLM has its own guardrails system (keyword filtering, PII detection, etc.), but it operates at the proxy level as middleware. Our three-layer governance — system prompt rewriting, `max_tokens` clamping, and tool restriction — modifies the actual LLM request parameters. This is not possible through LiteLLM's guardrail callbacks; it requires modifying the data dict before the call, which we do directly in our FastAPI endpoint.

Pillar 1: Guardrails

Guardrails are **declared** in `guardrails.yaml`, not buried in code. Each session can override the defaults.

Enforced guardrails

#	Guardrail	Declaration	Enforcement
G1	Daily token cap	<code>daily_cap:</code> <code>1_000_000</code> (user-configurable)	Calendar-day reset at midnight local time. Notify when exceeded; sessions continue. Portal shows "resets in Xh Ym."
G2	Session token cap	<code>session_cap:</code> <code>200_000</code>	Notify when session exceeds cap; agent continues
G3	Budget consumption zones	<code>zones: {green:</code> <code>60%, yellow:</code> <code>85%, red: 100%}</code>	Three-layer enforcement: system prompt rewrite,

#	Guardrail	Declaration	Enforcement
			max_tokens clamping, tool restriction — graduated per zone
G4	Loop detection	loop_threshold: 5	Same tool + same input hash repeated N times → LOOP_DETECTED alarm; notify user
G5	Session timeout	session_timeout: 30m	Wall-clock time exceeded → SESSION_TIMEOUT alarm; notify, save checkpoint
G6	Tool-category budget weighting	tool_category_limit: 50%	No single tool category may consume >50% of session budget → TOOL_CATEGORY_BIAS alarm; inject warning context

Enforcement model

The harness uses a **three-layer graduated enforcement** model. No layer hard-stops the agent, but each layer constrains behavior more tightly as the budget depletes — the agent *cannot* ignore the constraints.

Layer 1: Progressive system prompt rewrites

At each zone transition, the harness rewrites the agent's entire system prompt with progressively tighter instructions:

Zone	System prompt instruction
Green	"You have ample token budget. Be thorough: write tests, add documentation, explore alternatives."
Yellow	"Budget is limited. Prioritize the core task. Skip documentation and non-essential tests. Be concise."
Red	"Budget critical. Output only the final deliverable. No explanations, no exploration, no tests. If you cannot finish in two turns, deliver what you have."

Layer 2: Zone-based max_tokens limiting

The harness sets the max_tokens parameter on every API call based on the current zone. The agent physically cannot produce long responses in constrained zones:

Zone	max_tokens
Green	Full (configurable, default 4096)
Yellow	50% of green (2048)
Red	25% of green (1024)

Layer 3: Zone-based tool restrictions

The harness modifies the available `tools` list in the API call. In constrained zones, expensive or exploratory tools are removed:

Zone	Available tools
Green	All tools
Yellow	Remove <code>web_search</code> , <code>browser_*</code> (expensive exploration)
Red	Only <code>read_file</code> , <code>patch</code> , <code>terminal</code> (delivery-only tools)

The agent cannot search the web, browse, or delegate in red zone — those tools don't exist in the API call.

Alarms (always active)

In addition to the three enforcement layers, every guardrail also produces **alarm notifications**: structured alarms fired to the user via configured notification channel for visibility, even when the harness is automatically constraining the agent.

Declaration format

```
# guardrails.yaml
daily_cap: 1000000
session_cap: 200000
zones:
  green: 0.60 # 0-60%: normal operation
  yellow: 0.85 # 60-85%: context injection, wrap-up signal
  red: 1.0 # 85-100%: alarm fires
loop_threshold: 5
session_timeout: 1800 # seconds
tool_category_limit: 0.50
```

Per-session overrides are passed as JSON in the `/session/start` payload.

Pillar 2: Checkpoints

Checkpoints evaluate agent output at **session boundaries** using explicit pass/fail criteria. They read from the session artifact (token log, tool trace, guardrail state) — the agent itself is not evaluated directly.

Checkpoint types

#	Checkpoint	Pass criteria	Evaluates
C1	Budget health	Session consumed $\leq$ remaining daily budget * fairness factor	Was this session's spend proportional to work remaining?
C2	Stuck-loop score	Loop alarms fired < 3 during session	Did the agent get stuck in repetitive behavior?
C3	Tool diversity	Agent used ≥ 3 distinct tool categories	Was the agent's tool usage well-rounded, or did it rely on one thing?
C4	Timeout respect	Session ended before timeout (natural completion)	Did the agent finish its work before the clock ran out?

Persistence & replay

Every session produces a JSON artifact:

```
session_artifacts/  
  {session_id}/  
    token_log.json      # Every API call: timestamp, model, prompt_tokens,  
completion_tokens      # Every tool invocation: name, input_hash, timestamp,  
    tool_trace.json     # Every tool invocation: name, input_hash, timestamp,  
duration               # All alarms fired during session  
    alarms.json         # All alarms fired during session  
    guardrail_snapshots.json # Guardrail state at each turn boundary  
    checkpoint_results.json # Pass/fail + details for each checkpoint  
    output_summary.json  # Agent's final result
```

The checkpoint evaluator is a **stateless function**: `evaluate(session_artifact) → CheckpointResult[]`. To replay from any checkpoint, load the artifact and re-run the evaluator — no agent re-execution needed.

Pillar 3: Material Handling

Clean interfaces for passing material between the **user**, the **harness**, and the **agent**.

Interfaces

Interface	Direction	Transport	Description
Task submission	User → Harness	Portal (AGILE Board)	User creates task with description, token estimate. Board shows budget impact before dispatch.
Session dispatch	Harness → Agent	Proxy (API call forwarding)	Harness starts agent with guardrail config, budget zone, and task context injected as

Interface	Direction	Transport	Description
			system prompt.
Token analytics	Harness → User	Portal (Dashboard)	Charts: token burn over time, tool usage distribution, budget zone timeline, session history.
Session report	Harness → User	Portal + REST API	Per-session: token totals, tool call breakdown, alarms fired, checkpoint results, agent output.
Guardrail config	User → Harness	guardrails.yaml + REST API	Declare defaults; override per session.
Alarm notifications	Harness → User	Webhook / OS notification	Push alarms to configured channel (Discord, Slack, macOS notification).

AGILE Board

A visual task board with columns: **Backlog** → **Estimated** → **Running** → **Review** → **Done**. Each task card shows:

- Task description
- Estimated token budget (LLM-assisted, user-overridable)
- Recommended model (task-complexity driven, user-overridable)
- Actual token cost + estimate vs. actual delta (populated on completion)
- Session link

Token estimation (LLM-assisted): User types a task description. The harness calls a cheap model (e.g., GPT-4o-mini) with a prompt that explains the task and asks "how many tokens would a coding agent need to complete this?" The pre-filled estimate is displayed; user can override before dispatching.

Model recommendation (task-complexity matching): During estimation, the harness also classifies the task's complexity — simple, modest, or complex — and recommends a model tier:

- **Simple** (e.g., center a div, fix a typo) → Haiku / GPT-4o-mini
- **Modest** (e.g., add an API endpoint, write unit tests) → Sonnet / GPT-4o
- **Complex** (e.g., refactor auth, design a new feature) → Opus / GPT-4.5

The recommendation is pre-selected in a model dropdown; user can override. The harness also applies a **budget-aware clamp**: if remaining daily budget is below a threshold, it downgrades the recommendation one tier (complex → modest, modest → simple) and shows a note: "Budget is

tight — downgraded from Sonnet to Haiku."

Direct dispatch: One task → one session → one budget. "Run" starts a session immediately with the task description injected as the agent's first prompt, using the selected model. No queuing, no batching — cleanest mental model for analytics.

Pre-dispatch budget check: Before dispatch, the board checks whether the task's estimated tokens exceed the remaining daily budget. If they do, a warning banner appears ("⚠ This task's estimate (200K) exceeds remaining daily budget (100K). The DAILY_CAP_EXCEEDED alarm will fire during this session.") but the user can still dispatch — consistent with soft-enforcement across all guardrails.

Session completion lifecycle:

- Clean completion → card lands in **Done** with green badge, estimate vs. actual delta, and model used
- Checkpoint failure or alarm fired → card lands in **Review** with red badge; human must review session report and manually advance to Done

Global budget bar at top: "800,000 / 1,000,000 daily tokens (resets in 4h 32m)." Shows consumption across all sessions today. When estimating a new task, the bar previews the post-estimate utilization.

Midnight reset during active sessions: The zone state is always relative to the live daily budget, not locked at session start. If a session is in yellow or red zone and the clock hits midnight, the daily budget resets to zero consumption, the session returns to green zone, and the agent regains full tools and max_tokens. No session restart needed — the proxy re-evaluates zone on every request. The portal shows a notification: "Daily budget reset. Session returned to green zone." This proves the harness is stateful and responsive, not a one-time config check.

Second-worker swap (bonus): Completed task cards in the Done column include a "Re-run with..." action. User picks a different model, and the same task is re-dispatched to a new agent — zero harness changes. Portal shows both sessions side-by-side for comparison. This proves agent portability: the harness governs whichever agent is plugged in.

REST API surface

POST	/session/start	← Start a new agent session with guardrail overrides
GET	/session/{id}/report	← Full session analytics
GET	/session/{id}/artifact	← Raw JSON artifact (for checkpoint replay)
POST	/session/{id}/checkpoint/evaluate	← Re-run checkpoint evaluation
GET	/guardrails	← Current guardrail config
PUT	/guardrails	← Update guardrail defaults
POST	/tasks	← Create task on AGILE board
PUT	/tasks/{id}	← Update task (estimate, status)
POST	/estimate	← Estimate tokens + recommend model for a task description
GET	/dashboard	← Aggregate analytics (all sessions)
GET	/alarms	← Alarm history with filters
POST	/alarms/{id}/resolve	← Mark alarm as resolved

Pillar 4: Alarms

Alarms produce **structured output** with named types, context, severity, and recommended action.

Alarm taxonomy

Alarm Type	Trigger	Severity	Context	Recommended Action
BUDGET_YELLOW	Session enters yellow zone (>60% cap)	LOW	Current session spend, remaining budget, zone threshold	Inject wrap-up context into agent; no user action needed
BUDGET_RED	Session enters red zone (>85% cap)	MEDIUM	Current session spend, remaining budget, zone threshold	Notify user via configured channel; agent continues
DAILY_CAP_EXCEEDED	Daily token cap reached	HIGH	Daily total, per-session breakdown, timestamp	Notify user; sessions are not blocked — governance is visibility-only
LOOP_DETECTED	Same tool + same input hash repeated $\geq N$ times	MEDIUM	Tool name, input hash, repetition count, timestamps	Notify user; agent continues — human decides whether to intervene
SESSION_TIMEOUT	Wall-clock time exceeded	MEDIUM	Session duration, timeout threshold, last tool call	Notify user; save checkpoint; agent continues
TOOL_CATEGORY_BIAS	One tool category >50% of session budget	LOW	Category name, percentage, tool breakdown	Inject warning context into agent
CHECKPOINT_FAIL	Any checkpoint evaluated as fail	MEDIUM	Checkpoint name, fail reason, session ID	Escalate to human for review; halt task pipeline until resolved

Alarm structure

Every alarm is a JSON object:

```
{
  "id": "alarm_abc123",
  "type": "BUDGET_RED",
  "severity": "MEDIUM",
  "session_id": "sess_xyz789",
  "timestamp": "2026-06-12T14:30:00Z",
  "context": {
    "current_spend": 178000,
    "session_cap": 200000,
    "zone": "red",
```

```

    "percentage": 0.89
  },
  "recommended_action": "Notify user; agent continues. Review session in
portal."
}

```

Human-in-the-Loop Escalation

The harness does not make decisions for the human. It surfaces what it knows, recommends an action, and defers. The human always has the final say — this is by design, not a limitation.

Escalation paths

Path	Trigger	Harness action	Human decision
Budget override	Task estimate exceeds remaining daily budget	Warning banner on AGILE Board: "Estimated 200K exceeds remaining 100K. The daily cap alarm will fire."	Override and dispatch anyway, reduce the estimate, or cancel
Daily cap exceeded	DAILY_CAP_EXCEEDED alarm fires mid-session	Notification + "Raise budget" button in portal	Raise the cap, abort session, or let it continue
Loop detected	LOOP_DETECTED alarm fires	Notification: "Agent repeated read_file(path=X) 8 times."	Continue (agent is fine), abort session, or adjust guardrail threshold
Session timeout	SESSION_TIMEOUT alarm fires	Notification + saved checkpoint	Continue, abort, or extend timeout
Checkpoint failure	Session ends with failed checkpoint	Card lands in Review column; pipeline halts	Review session report → manually advance to Done or re-dispatch
Zone escalation	Session enters yellow/red zone	Context injection + tool restrictions applied automatically; alarm logged	No action required — harness handles this. Human can override zone if needed

The principle

The harness constrains the **agent** (via system prompt, max_tokens, tool restrictions). It does not constrain the **human**. When the harness detects a problem, it tells you — clearly, with context and a recommendation — and you decide. This is the "stop and ask rather than guess" requirement from the challenge spec.

Portal action buttons

Every alarm in the portal includes action buttons. The harness polls for the human's response and acts on it:

Button	What it does
Continue	Dismiss alarm; agent keeps running with current constraints
Abort session	Terminate the agent process; session ends; artifacts saved
Raise budget	Update the daily cap; session re-evaluates zone and may return to green
Adjust guardrail	Modify the guardrail threshold (e.g., loop_threshold 5 → 10); changes take effect on the next request

Agent Interface (Swappable)

The harness is agent-agnostic. Any coding agent that speaks the OpenAI-compatible API can be governed. Here is the concrete integration with **Hermes** (and any agent that follows the same pattern).

How the agent connects

The agent is pointed at the harness URL instead of directly at the LLM provider. The harness URL can be `localhost:8000` (local development) or `https://harness.fly.dev` (cloud deployment). The agent does not know or care — it sends standard HTTP requests.

```
# Hermes points at the harness
OPENAI_BASE_URL=http://localhost:8000/v1
OPENAI_API_KEY=*** # harness-generated per-session key
```

Session dispatch (when the user clicks "Run")

1. **Harness creates a session record** in SQLite: `session_id`, task description, model, budget, guardrail overrides.
2. **Harness generates a session-scoped API key** (e.g., `sk-harness-a1b2c3d4`). This key maps to the session — all requests carrying it are governed under that session's budget.
3. **Harness launches the agent as a subprocess**, passing the harness URL and session key as environment variables:

```
# Inside the harness FastAPI app
subprocess.Popen(
    ["hermes", "--prompt", task.description],
    env={
        "OPENAI_BASE_URL": "http://localhost:8000/v1",
        "OPENAI_API_KEY": f"sk-harness-{session.id}",
        # Provider API key is NOT exposed to the agent –
        # the harness injects it when forwarding via LiteLLM
    },
)
```

1. **Portal begins polling** `GET /session/{id}/report` for live updates.

Per-request governance flow

On every API call from the agent:

```
Hermes: POST /v1/chat/completions
Header: Authorization: Bearer sk-harness-a1b2c3
Body: {model: "claude-haiku", messages: [...], tools: [...], max_tokens: 4096}

↓
Harness (FastAPI endpoint):
1. Look up session "sess_a1b2c3" from API key
2. Evaluate live daily budget: 350,000 / 1,000,000 used → zone = green
3. Layer 1: rewrite system prompt – "Be thorough: write tests..."
4. Layer 2: max_tokens stays 4096 (green zone)
5. Layer 3: tools unchanged (green zone – all tools available)
6. Call LiteLLM:
    response = await litellm.acompletion(
        model="claude-haiku",
        messages=messages,           # rewritten
        max_tokens=4096,             # clamped per zone
        tools=tools,                 # filtered per zone
        stream=True,
        stream_options={"include_usage": True},
    )
7. LiteLLM forwards to real Anthropic API, returns normalized response
8. Extract token counts: response.usage.prompt_tokens=2150,
completion_tokens=850
9. Update session artifact: token_log.append({turn: 3, prompt: 2150,
completion: 850})
10. Update daily counter: 350,000 → 353,000
11. Return response to agent (usage field stripped – the agent doesn't need
it)
```

Later, at >60% daily budget (yellow zone):

The same flow, but:

- Step 3: system prompt → "Budget is limited. Prioritize core task. Be concise."
- Step 4: max_tokens → 2048
- Step 5: tools → web_search, browser_* removed

The agent cannot call those tools because they don't appear in the API response. The agent cannot produce long output because max_tokens is clamped. The only behavior change the agent "sees" is a different system prompt, but the harness constrains it at the transport level.

At >85% daily budget (red zone):

- System prompt → "Budget critical. Output only the final deliverable."
- max_tokens → 1024
- Tools → only read_file, patch, terminal

Data plane vs. control plane

Plane	Endpoint	Caller	Frequency
Data plane	POST	The agent (Hermes)	Every API call — high

Plane	Endpoint	Caller	Frequency
	/v1/chat/completions		volume, streaming
Control plane	POST /session/start, GET /dashboard, etc.	The portal (user)	On user action — low volume, request/response

Both run in the same FastAPI process. The data plane is the governance hot path; the control plane serves the user experience.

Swapping agents

The same mechanism works identically for any OpenAI-compatible agent:

```
# Claude Code
claude --api-base http://localhost:8000/v1 --api-key sk-harness-***

# Codex CLI
codex --base-url http://localhost:8000/v1 --api-key sk-harness-***

# Any OpenAI SDK client
client = OpenAI(base_url="http://localhost:8000/v1", api_key="sk-harness-***")
```

Each agent gets its own session key. The harness governs identically regardless of which agent produced the request. LiteLLM handles provider translation — the agent can request `claude-opus` (Anthropic-native) or `gpt-4o` (OpenAI) and the harness normalizes both through the same code path. The portal shows all sessions side-by-side for comparison, enabling the second-worker swap demo beat.

Tech Stack

Layer	Technology	Rationale
Runtime	Python 3.11+	Fast proxy, rich ecosystem
Framework	FastAPI	Async-native, OpenAPI docs, Pydantic validation
Database	SQLite	Zero setup, single file, survives Docker restarts via volume
Portal UI	HTMX + Jinja2 + Chart.js	Server-rendered, no bundler, interactive via HTMX
Charts	Chart.js	Token burn timeline, tool distribution pie
Notifications	Discord/Slack webhook + macOS osascript	Push alarms to user
Container	Docker (single Dockerfile)	Identical local and cloud deployment
Cloud	Fly.io / Railway	Fast deploy, free tier sufficient

Layer	Technology	Rationale
		for demo